PRÁCTICA 01 · ENCARGO PROFESIONAL

SmartCampus

Diseña y demuestra un pequeño servicio de datos

DURACIÓN	MODALIDAD	ENTREGA
120 minutos	Grupos*	Repositorio + evidencias

*Según las indicaciones del profesorado para esta sesión.

USO DE IA · NIVEL AMARILLO
IA permitida con trazabilidad. Es obligatorio entregar la memoria con capturas de todos los prompts utilizados (véase apartado 07).

01 El encargo

Una empresa de gestión de edificios universitarios quiere construir un prototipo para consultar la información que registran sus sensores ambientales. Dispone de mediciones en formato CSV y necesita dejar de depender de un fichero aislado: quiere almacenar las lecturas en una base de datos, analizarlas desde un notebook y consultar o registrar nuevas mediciones mediante una API.

Vuestro papel: sois el equipo encargado de entregar una primera versión funcional, reproducible y verificable del sistema. No se pide una web ni una interfaz gráfica.

02 Objetivo de la práctica

Integrar los componentes trabajados en clase dentro de un mismo flujo de datos. Al terminar, vuestra solución deberá permitir:

- Levantar tres servicios relacionados mediante Docker Compose.
- Cargar y consultar las lecturas utilizando pandas y SQLAlchemy.
- Cambiar de PostgreSQL a SQLite en el notebook modificando la conexión, sin reescribir la lógica de trabajo.
- Exponer y consumir una API Flask que lea y escriba en PostgreSQL.

Arquitectura de referencia



EL RESULTADO ESPERADO
Una llamada POST a la API añade una medición; desde Jupyter podéis comprobar después que esa medición existe en PostgreSQL. Esa prueba de extremo a extremo es el núcleo del taller.

03 Requisitos técnicos y funcionales

Los siguientes requisitos son obligatorios. Se describe qué debe conseguir la solución y qué aspectos debéis tener en cuenta, pero no se prescribe una implementación única.

Reto A · Infraestructura con Docker Compose

- **Tres servicios:** un entorno Jupyter, una base de datos PostgreSQL y una API Flask. Elegid nombres claros para los servicios (por ejemplo, jupyter, db y api).
- **Configuración y conectividad:** los contenedores deben comunicarse dentro de Compose; exponed al equipo anfitrión los puertos necesarios para acceder a Jupyter y a la API.
- **PostgreSQL:** debe definirse el nombre de la base de datos, el usuario y la contraseña mediante las variables de entorno POSTGRES_DB, POSTGRES_USER y POSTGRES_PASSWORD.
- **Persistencia:** los datos de PostgreSQL deben almacenarse en un volumen para no perderse al reiniciar los contenedores.
- **Dependencias:** Jupyter ha de disponer de pandas, SQLAlchemy, un controlador para PostgreSQL y requests; la API tendrá Flask y lo necesario para conectarse a PostgreSQL.

RECORDATORIO ÚTIL

Para acceder de un contenedor a otro dentro de Compose, pensad en el nombre del servicio de destino. «localhost» hace referencia al propio contenedor, no a los demás.

Reto B · Carga, almacenamiento y análisis

Trabajaréis con un fichero sensores.csv proporcionado para la actividad. Cada fila representa una lectura ambiental tomada en un edificio universitario.

Campo	Significado	Tipo esperado
edificio	Identificador del edificio	Texto (A, B, C...)
fecha_hora	Instante en que se tomó la lectura	Fecha y hora
temperatura	Temperatura ambiente	Numérico, °C
humedad	Humedad relativa	Numérico, %
co2_ppm	Concentración de CO_2	Entero, ppm

Desde vuestro notebook de Jupyter debéis:

- Leer el CSV con pandas (recordad la función pd.read_csv), revisar columnas y tipos, y preparar los datos para su almacenamiento.
- Crear una conexión con SQLAlchemy y cargar las lecturas en una tabla llamada lecturas en PostgreSQL.
- Reutilizar la lógica de carga y consulta para SQLite, sustituyendo únicamente el engine o la URI de conexión. Podéis encapsular la lógica común en funciones.
- Mostrar cuántos registros se han cargado en cada motor y obtener al menos dos resultados: temperatura media por edificio y edificio con mayor CO_2 medio.

Debe apreciarse el uso de pandas y SQLAlchemy. Para las consultas podéis combinar pd.read_sql con operaciones de pandas o consultas SQL, según lo visto en clase.

Reto C · API de datos con Flask

La empresa quiere consultar los datos sin abrir un notebook y enviar lecturas nuevas desde un dispositivo externo. Construid una API REST mínima que trabaje sobre la misma base PostgreSQL utilizada en el reto anterior.

Método y ruta	Qué debe hacer	Resultado verificable
GET /health	Comprobar que el servicio responde y que puede acceder a la base de datos.	HTTP 200 + respuesta JSON de estado correcto.
GET /lecturas	Consultar las lecturas almacenadas en PostgreSQL.	HTTP 200 + una colección JSON con las lecturas.
POST /lecturas	Recibir una nueva lectura JSON y guardarla en PostgreSQL.	HTTP 201 + confirmación JSON; el registro queda persistido.

El JSON recibido por POST debe utilizar los mismos cinco campos del CSV. Por ejemplo:

```
{
  "edificio": "A",
  "fecha_hora": "2026-10-13 11:30:00",
  "temperatura": 23.5,
  "humedad": 51,
  "co2_ppm": 1250
}
```

- El endpoint GET debe recuperar datos reales, no devolver una lista codificada en Python.
- El endpoint POST debe insertar en la misma tabla lecturas que utilizó el notebook.
- Las respuestas deben enviarse en JSON. Ante una petición POST sin campos obligatorios, devolved un error HTTP 400 con una explicación comprensible.

¿CÓMO PODÉIS PROBAR LA API?

GET puede comprobarse desde el navegador, Postman o un notebook. Para POST podéis utilizar Postman o enviar una petición HTTP con la biblioteca requests desde Python. El navegador por sí solo no envía POST al introducir una URL.

Reto D · Demostración de extremo a extremo

La prueba final debe hacerse desde Jupyter. Debéis:

- Consultar el número de filas existentes en PostgreSQL o registrar un valor de referencia antes de la prueba.
- Enviar con requests una petición POST /lecturas con una medición nueva y reconocible.
- Comprobar que la API devuelve el código esperado y consultar después PostgreSQL con SQLAlchemy para localizar la nueva medición.
- Mostrar en el notebook las evidencias de la petición, la respuesta y la lectura recuperada desde la base de datos.

Importante: se evaluará la conexión real entre servicios. Una captura de JSON sin persistencia efectiva no demuestra que la integración funcione.

04 Entrega y evidencias

Entregad un enlace al repositorio GitHub que contenga el proyecto completo, ejecutable y suficientemente documentado. El repositorio debe ser accesible para su corrección; podéis usar uno público o facilitar acceso al profesorado.

Contenido mínimo del repositorio

- docker-compose.yml y los Dockerfile o archivos de configuración necesarios.
- Código fuente de la API Flask y archivos de dependencias.
- Un notebook .ipynb con lectura del CSV, carga en PostgreSQL y SQLite, análisis y prueba final de integración.
- El CSV utilizado, una carpeta evidencias/ con capturas, un README.md de ejecución y la memoria de IA del apartado 07 (o declaración de no uso).

Cuatro evidencias obligatorias

Evidencia	Qué debe verse en la captura
01 · Servicios	Los tres servicios en ejecución (por ejemplo, salida de docker compose ps).
02 · Bases de datos	Notebook con las consultas y recuentos de registros en PostgreSQL y SQLite.
03 · API	Respuesta JSON a GET /lecturas, y ejecución satisfactoria de POST /lecturas.
04 · Integración	En Jupyter, POST realizado y comprobación de esa misma lectura en PostgreSQL.

README: qué debe explicar

Incluid una descripción breve de la solución, servicios utilizados, requisitos previos, instrucciones para levantar Compose, cómo abrir y ejecutar el notebook, cómo probar los endpoints y cómo detener los servicios. Especificad las variables de entorno necesarias, sin publicar credenciales reales.

05 Organización del trabajo

Tramo orientativo	Objetivo
0–20 min	Preparar Compose y comprobar conectividad entre contenedores.
20–55 min	Importar el CSV, probar PostgreSQL/SQLite y obtener los análisis.
55–85 min	Construir y probar los tres endpoints de Flask.
85–110 min	Ejecutar la prueba de integración completa.
110–120 min	Comprobar evidencias, cerrar la memoria de IA y subir el repositorio (registrad los prompts durante toda la sesión).

06 Evaluación

Se valorará la funcionalidad real, la integración entre componentes y la posibilidad de reproducir el proyecto a partir del repositorio. Las capturas complementan la entrega, pero no sustituyen el código.

Criterio	Qué se comprobará	Puntos
Infraestructura	Compose con tres servicios, comunicación y persistencia de PostgreSQL.	2,0
Datos y portabilidad	Carga de lecturas y análisis con pandas/SQLAlchemy en PostgreSQL y SQLite sin duplicar la lógica.	2,0
API Flask	GET /health, GET /lecturas y POST /lecturas conectados a PostgreSQL, respuestas HTTP/JSON y validación mínima.	2,0
Integración	Petición POST desde Jupyter y verificación de la lectura insertada desde SQLAlchemy.	2,5
Entrega	Repositorio, README reproducible, capturas técnicas y memoria de uso de IA con trazabilidad completa (o declaración de no uso).	1,5

EXTENSIÓN OPCIONAL

Si terminái antes, podéis añadir GET /resumen para devolver indicadores agregados por edificio (temperatura media, CO_2 medio y número de lecturas que superan 1.000 ppm). No es necesario para obtener la máxima calificación.

Lista de comprobación antes de entregar

- ☐ ¿Los tres servicios arrancan usando docker compose up --build?
- ☐ ¿El notebook puede conectarse a PostgreSQL y a SQLite?
- ☐ ¿Se obtienen los resultados analíticos solicitados?
- ☐ ¿Los tres endpoints de Flask responden como se espera?
- ☐ ¿El POST desde Jupyter crea un registro verificable en PostgreSQL?
- ☐ ¿Un compañero podría reproducir el proyecto siguiendo solo el README?
- ☐ ¿Habéis incluido capturas y evitado subir contraseñas reales?
- ☐ ¿Habéis documentado todos los prompts de IA, también los que fallaron, o declarado que no usasteis IA?

Uso de recursos y aclaraciones

Podéis consultar los materiales vistos en clase y la documentación oficial de Docker, pandas, SQLAlchemy y Flask. La actividad es de nivel AMARILLO: podéis utilizar IA, pero debéis verificar su resultado y documentar todas las interacciones según el apartado 07.

CRITERIO DE CIERRE

El objetivo no es crear una plataforma de producción: es entregar un prototipo pequeño pero funcional, donde la infraestructura, el tratamiento de datos y la API encajen entre sí.

07 Uso de IA · NIVEL AMARILLO

En esta práctica está permitido utilizar herramientas de inteligencia artificial, tanto para consultar conceptos como para apoyar el desarrollo o la depuración del proyecto. **El uso exige trazabilidad y validación propias.**

Memoria obligatoria de uso de IA

Además del repositorio y de las evidencias técnicas, debéis incluir un documento **memoria_uso_IA.pdf** (o DOCX) que recoja el proceso real de trabajo con IA. La memoria será una parte obligatoria de la entrega cuando se haya usado IA.

• **Herramientas y finalidad.** Indicad qué asistentes/modelos habéis utilizado, para qué tareas (Docker, SQL, Flask, depuración, redacción, etc.) y, en trabajos en grupo, quién los utilizó.

• **Capturas de TODOS los prompts.** Incorporad capturas de pantalla legibles y ordenadas cronológicamente de cada prompt enviado a la IA, también reformulaciones, reintentos y prompts que no sirvieron. Mostrad la respuesta correspondiente o la parte necesaria para comprender el resultado. No se admite una selección solo de los mejores.

• **Qué funcionó y qué no.** Para cada interacción o secuencia de trabajo, explicad brevemente qué se pretendía, qué propuesta de la IA fue útil y qué respuestas fallaron, contenían errores o no se utilizaron.

• **Correcciones y validación.** Indicad qué modificasteis vosotros y cómo contrastasteis el resultado: ejecución del código, errores detectados, consultas SQL, respuesta HTTP, verificación de persistencia u otras pruebas.

• **Balance final.** Cerrad con una reflexión breve sobre el valor y las limitaciones reales de la IA en vuestro proyecto: dónde ahorró trabajo, dónde generó problemas y qué decisiones fueron del equipo.

Condiciones y responsabilidades

• Guardad las capturas durante la sesión, no reconstruyáis después una conversación idealizada. Numeradlas para que puedan relacionarse con la parte del código o el reto correspondiente.

• No subáis contraseñas, tokens ni datos personales o confidenciales a las herramientas de IA, al repositorio o a las capturas; ocultad esos datos sin tapar el prompt ni la evidencia académica.

• Todo el código y los resultados entregados deben poder ejecutarse, comprenderse, verificarse y defenderse por el equipo, aunque se hayan producido con ayuda de IA.

• Si no habéis usado IA, incluid un archivo con una declaración expresa de «No se han utilizado herramientas de IA en esta práctica». En ese caso, no son necesarias capturas.

• La omisión, inexactitud o falsedad en la trazabilidad exigida se tratará según el Reglamento de uso de IA y la normativa académica de la Universidad Europea.

08 Retos bonus · ampliaciones opcionales

Si terminái los retos obligatorios y podéis demostrar su funcionamiento, podéis ampliar el prototipo con estas mejoras. **No son necesarias para obtener la máxima calificación de la práctica.**

BONUS 01 · GET /resumen

Implementad un endpoint que devuelva indicadores agregados por edificio: temperatura media, CO_2 medio y número de lecturas que superan 1.000 ppm.

Evidencia: llamada a /resumen y contraste de al menos un valor con el notebook.

BONUS 02 · Filtrado de lecturas

Ampliad GET /lecturas para aceptar filtros opcionales mediante la URL, por ejemplo /lecturas?edificio=A, sin alterar el comportamiento original cuando no hay filtros.

Evidencia: misma llamada con y sin filtro; las respuestas deben ser coherentes con los registros almacenados.

BONUS 03 · Alertas de CO_2

Añadid GET /alertas?umbral=1000 para devolver solo las lecturas cuyo CO_2 supere el umbral indicado. Debe funcionar con un valor configurable.

Evidencia: peticiones con dos umbrales diferentes y resultados verificables frente a la base de datos.

BONUS 04 · Validación avanzada del POST

Reforzad POST /lecturas para rechazar tipos o valores incoherentes (por ejemplo, humedad fuera de 0–100, CO_2 negativo o fecha inválida), además de los campos ausentes.

Evidencia: una petición válida y varias inválidas, con HTTP 400 y mensajes comprensibles; ninguna inválida debe persistir.

BONUS 05 · Pruebas automáticas

Preparad un pequeño script o batería de pruebas (por ejemplo, con requests o pytest) que compruebe /health, lectura, inserción, validación de errores y persistencia de datos.

Evidencia: ejecución reproducible de las pruebas, resultado y breve explicación en el README.

IMPORTANTE Documentad cualquier bonus realizado en el README y aportad evidencias de ejecución. Los bonus no sustituyen ninguno de los requisitos obligatorios ni la memoria de IA.